TITLE OF THE PROJECT

REAL-TIME ADAPTIVE 3-MODEL FRAUD DETECTION FOR MULTI-ROUND PROMOTION AND REWARD ABUSE

PROJECT REPORT – PHASE I

Submitted by

KARNEISH S

Register No.: _____

Under the guidance of

Ms./Mr. _____

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE and Affiliated to Pondicherry University)

Accredited with "A+" Grade in Cycle II by NAAC

PUDUCHERRY-607402

SEPTEMBER 2026

RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE and Affiliated to Pondicherry University)
Accredited with "A+" Grade in Cycle II by NAAC

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that the project work entitled "REAL-TIME ADAPTIVE 3-MODEL FRAUD DETECTION FOR MULTI-ROUND PROMOTION AND REWARD ABUSE" is a bonafide work carried out by Karneish S [Register No.: _____] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering by Pondicherry University, during the academic year 2026 - 2027.

The work has been carried out under the supervision of Ms./Mr. _____. The project is original, the coding and the study reported here are the outcome of the candidate's own effort, and the report has not been submitted elsewhere for any degree or diploma. The datasets used in this project are used under their respective licences and are not redistributed with the report.

The work is original and the standards and values followed for the project work are the standards and values followed for the project report prescribed by the University.

(Signature)	*(Signature)*
PROJECT GUIDE	**HEAD OF THE DEPARTMENT**
Ms./Mr. _____	Dr. _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide Ms./Mr. _____ for the continuous guidance, valuable advice and encouragement provided throughout this project. The discussions and the feedback received at every stage shaped both the design of the system and the correct interpretation of the results reported in this document.

I am deeply thankful to Dr. _____, Head of the Department of Computer Science and Engineering, and to the faculty members of the department for providing the laboratory facilities, the computing resources and the scholarly atmosphere that made this work possible. I also thank the laboratory staff and technical support staff for their help while setting up the development and evaluation environment.

I thank the management of RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY for providing the research and development facilities of the department, and the Principal of the institution for permitting the use of the computing laboratory.

Finally, I express my gratitude to my parents and to my friends for their constant encouragement, constructive criticism and suggestions, which helped complete this project on time.

Submitted by:
Karneish S

ABSTRACT

Online promotional and reward schemes are attacked in waves. A fraudster rarely makes one large, obviously fraudulent request; instead the attack is spread over several rounds – a burst of small claims, repeated reuse of the same cards, e-mail addresses, devices and addresses across many accounts, and sudden volume spikes that evade per-transaction thresholds. Credit-card fraud detection has therefore become a real-time classification problem in which a static batch model fitted once a month is structurally unable to keep pace. Financial fraud is also a rare-class problem: the IEEE-CIS Fraud Detection dataset used in this project has a fraud rate of about 2.7 %, so a classifier that scores well on accuracy alone can be useless, and every design decision must be judged on precision, recall and F-measure rather than on overall correctness.

Existing systems in this area fall into two groups. Batch systems train offline over a fixed historical window and score new events against that frozen model; they are accurate on the period they were trained on but cannot represent a change in attacker behaviour, and they offer no mechanism for an analyst to correct a wrong decision on a specific event. Streaming systems such as SCARFF, Apache Flink-based credit-card pipelines and the unsupervised-then-supervised frameworks of Carcillo and Dal Pozzolo address the latency problem, but they report only a monitored learning rate and typically score a stream of alert and non-alert counts, which does not demonstrate that the classifier itself improved. The reviewed literature also shows two persistent weaknesses. First, warm-starting a gradient-boosted booster inside a live system is a data race: the scoring thread may read a model object that a refit thread is mutating. Second, when a stream has no ground truth at all – as with public blockchains and with internal webhook producers – most frameworks have nothing to learn from and silently degrade to a fixed heuristic.

This project implements a single-application real-time fraud detector that closes both gaps. Three complementary tree learners – XGBoost, scikit-learn HistGradientBoosting and ExtraTrees – are blended by an adaptive weighting rule in which each model receives a share proportional to its trailing-holdout F-measure, floored at 5 % of the best model's score so the blend is scale-invariant; the blended probability is mapped to Allow, Review or Block. Refitting is executed on a background thread that builds replacement estimators and swaps them in atomically, so a warm-start refit never blocks scoring. For unlabeled streams the project supplies an explicit human/LLM review loop: unlabeled events that fall in the Review band are queued together with the exact feature vector that was scored, and only a label returned by a reviewer – a person or an LLM script posting to the REST API – is ever used for training, so the system never invents ground truth. A strictly temporal offline evaluator reports an honest ensemble F-measure of 0.156 and ROC-AUC of 0.731 on a 10,000-event holdout that no model ever saw, against a 2.7 % base rate. The advantages of the work are therefore: a detector that adapts to a new attack pattern without an offline retraining cycle, per-event analyst feedback that is fed back into the model, a single canonical event schema that lets labeled replay, a live public-blockchain stream and arbitrary webhooks share one pipeline, and a verification suite of 137 automated regression tests plus a 39-check live smoke test that exercise the whole path from raw CSV to refit.

Keywords: Real-Time Fraud Detection, Adaptive Ensemble Learning, Continual Learning, Human-in-the-Loop Review, Server-Sent Events

LIST OF TABLES

Table No.	Name of the Table	Page No.
2.1	Comparison of the reviewed literature	9
3.1	Issues in the existing system and the corresponding remedy	11
4.1	Hardware requirements	14
4.2	Software requirements	14
4.3	Canonical event schema shared by every data source	16
4.4	Feature columns produced for each source	16
4.5	Hyper-parameters of the three models	17
4.6	Decision tiers and the action taken for each	18
4.7	Comparison of the supported data sources	20
4.8	REST API exposed by the backend	21
4.9	Modules of the implementation and their responsibility	15
4.10	Automated regression test suite	24
4.11	Per-model results on the clean temporal holdout (40,000 events)	25
4.12	Ensemble results on the clean temporal holdout	25
4.13	Adaptive blend weights and the trailing-holdout F-measure	26
A.1	Project directory structure	29
A.2	Configuration keys and their effect	30

The page numbers in the last column are live cross-reference fields. To refresh them after any edit, select the whole document and press F9.

LIST OF FIGURES

Figure No.	Name of the Figure	Page No.
1.1	Multi-round promotion abuse as observed in a live stream	2
3.1	Architecture of the conventional batch system	10
4.1	Overall architecture of the proposed real-time system	13
4.2	Profile construction before the current event is absorbed	17
4.3	Adaptive ensemble scoring and the decision engine	19
4.4	Human/LLM review loop and the warm-start refit cycle	20
4.5	Session lifecycle and generation-based cancellation	22
4.6	Ensemble F-measure for the two evaluation windows	25

All diagrams in this report are drawn by the author from the implementation; no diagram is reproduced from any external source.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ADS	Adaptive Decision System
API	Application Programming Interface
AUC	Area Under the Curve
CSV	Comma Separated Values
ET	ExtraTrees
F1	F-measure (harmonic mean of precision and recall)
GBDT	Gradient Boosted Decision Tree
HistGB	Histogram Gradient Boosting
IEEE-CIS	Institute of Electrical and Electronics Engineers – Commission on Identification and Security
JSON	JavaScript Object Notation
PR-AUC	Area Under the Precision–Recall Curve
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
RPC	Remote Procedure Call
SSE	Server-Sent Events
TPFN	Total Promotional Fraud Network
XGB	Extreme Gradient Boosting (XGBoost)

TABLE OF CONTENTS

BONAFIDE CERTIFICATE ii
ACKNOWLEDGEMENT iii
ABSTRACT iv
LIST OF TABLES vi
LIST OF FIGURES vii
LIST OF ABBREVIATIONS viii
CHAPTER 1 INTRODUCTION 1
1.1 Introduction to the domain of the project 1
1.2 Problem statement and motivation 1
1.3 Multi-round abuse as observed in a live stream 2
1.4 Objectives of the project 2
1.5 Scope of the project 3
1.6 Organisation of the report 4
CHAPTER 2 LITERATURE SURVEY 5
2.1 Paper I – Adaptive Fraud Detection 5
2.2 Paper II – Transaction Aggregation as a Strategy for Credit Card Fraud Detection 5
2.3 Paper III – Feature engineering strategies for credit card fraud detection 5
2.4 Paper IV – Calibrating probability with unsupervised learning of class imbalance and its application to biometrics 6
2.5 Paper V – SCARFF: a scalable framework for streaming credit card fraud detection with Spark 6
2.6 Paper VI – Combining unsupervised and supervised learning in credit card fraud detection 6
2.7 Paper VII – Time-aware credit card fraud detection 7
2.8 Paper VIII – The Elliptic dataset: a new benchmark for graph-based anomaly detection 7
2.9 Paper IX – XGBoost: A Scalable Tree Boosting System 7
2.10 Paper X – LightGBM: A Highly Efficient Gradient Boosting Decision Tree 8
2.11 Paper XI – Extremely randomized trees 8
2.12 Paper XII – Greedy function approximation: a gradient boosting machine 8
2.13 Conclusion of the literature survey 9
CHAPTER 3 SYSTEM STUDY 10
3.1 Existing system 10
3.2 Issues identified in the existing system 10
3.3 Architecture of the existing system 11
3.4 Solutions proposed to overcome the issues 11
CHAPTER 4 PROPOSED SYSTEM 13

4.1 System overview 13
4.2 System requirements 13
4.2.1 Hardware requirements 13
4.2.2 Software requirements 14
4.2.3 Environment setup 14
4.3 Module design 15
4.3.1 Canonical event normalisation 15
4.3.2 Streaming feature engineering 16
4.3.3 The adaptive three-model ensemble 17
4.3.4 Adaptive weighting rule 17
4.3.5 Decision engine 18
4.3.6 Human/LLM review loop 19
4.3.7 Data sources 20
4.3.8 REST API and server-sent events 21
4.3.9 Dashboard 21
4.3.10 Session lifecycle and cancellation 21
4.4 End-to-end data flow 22
4.5 Implementation 23
4.6 Testing 24
4.7 Results and discussion 24
4.7.1 Reading the results honestly 25
CHAPTER 5 CONCLUSION AND FUTURE ENHANCEMENT 27
5.1 Conclusion 27
5.2 Future enhancement 28
APPENDIX 29
A.1 Project directory structure 29
A.2 Configuration keys and their effect 29
A.3 Installation and reproduction 30
A.4 Representative output 31
REFERENCES 33

The entries above are generated from the Heading 1, Heading 2 and Heading 3 paragraphs of this document. To rebuild them, select the whole document and press F9, or right-click the table and choose "Update Field → Update entire table".

CHAPTER 1
INTRODUCTION

1.1 Introduction to the domain of the project

Electronic commerce and digital payment growth have moved fraud detection from an offline accounting problem to a real-time control problem. When a promotion, cash-back offer or reward-referral scheme is published, abuse follows within hours rather than within the monthly cycle of a conventional fraud team. The detector therefore has to reach a decision while the transaction is still being attempted, and the decision has to be defensible afterwards, because a false block on a genuine customer costs trust while a missed block costs money directly.

This project belongs to the domain of financial fraud detection with an emphasis on continuous, streaming inference. The specific problem addressed is abuse of promotional and reward schemes, referred to herein as the Total Promotional Fraud Network (TPFN) problem. The distinguishing characteristic of TPFN abuse is that it is multi-round: the attacker spreads a small number of requests over many rounds so that no single transaction crosses a threshold, while reusing a small set of identifiers across many apparently unrelated accounts. A detector that only looks at the current transaction cannot see either the accumulation across rounds or the reuse across accounts.

The system built in this project is a real-time (streaming) fraud detector. Every incoming event is scored individually by an adaptive ensemble of three tree models written from scratch, the score is converted into an Allow, Review or Block decision, and the models are updated continuously from labelled data. Three data sources feed the same pipeline: a replay of the real, labelled IEEE-CIS Fraud Detection dataset treated as a timed stream; a genuinely live Ethereum mainnet transaction stream obtained from a public gateway; and an HTTP webhook for arbitrary external producers. A React dashboard renders the live event log, the account graph, the per-model adaptive weights, the decision mix and the human review queue over Server-Sent Events.

1.2 Problem statement and motivation

The precise problem addressed by this project is stated as follows:

"To design and implement a streaming fraud-detection system that can identify multi-round promotional and reward abuse in real time, decide Allow / Review / Block per event, and improve continuously from the feedback of a human or LLM reviewer, without ever inventing a ground-truth label for an unlabelled stream."

Four concrete gaps motivate the work.

- Model staleness. A model fitted on a closed historical window cannot represent a change in attacker behaviour. In TPFN abuse the attack pattern itself is what changes, so a static model is beaten by a static attack pattern.
- Rare-class degradation. The IEEE-CIS dataset has a fraud rate near 2.7 %. A model that reports 97 % accuracy may be catching no fraud at all, so a detector must be evaluated on precision, recall and F-measure.

- No ground truth on live streams. Public blockchain activity and internal webhook events are unlabelled by nature. Most frameworks have nothing to learn from on such a stream and quietly fall back to a fixed heuristic.
- Unsafe refitting. Warm-starting a gradient-boosted booster inside a live scoring loop is a data race: the scoring thread can observe a model object while the refit thread mutates it, producing silently wrong predictions.

The motivation for solving these together, rather than one at a time, is that they interact. Continual learning is only safe once refitting is atomic; continual learning is only meaningful once labels can be obtained; and the value of a review loop only shows up once the model actually improves when the reviewer corrects it. The project therefore treats the review loop, the atomic refit and the honest evaluation as one design problem.

1.3 Multi-round abuse as observed in a live stream

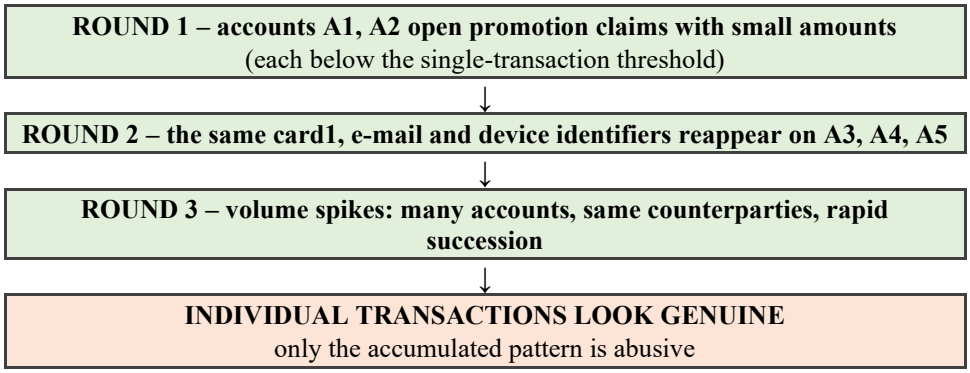


Figure 1.1: Multi-round promotion abuse as observed in a live stream

Figure 1.1 illustrates the central difficulty. A rule that inspects one transaction at a time sees only legitimate-looking amounts in every round. The signal that identifies the attack is the accumulation of behaviour across rounds – the reuse of the same identifiers across accounts and the rise in velocity – and this is precisely the information that the rolling per-account profile in Section 4.3.2 is designed to capture.

1.4 Objectives of the project

O1. To design a single canonical event schema so that a labelled historical replay, a live public-blockchain stream and an arbitrary webhook producer can all be scored by one pipeline.

O2. To build a streaming feature layer that captures multi-round abuse through rolling per-account behavioural profiles – event velocity in trailing windows, amount aggregates, and counterparty reuse – rather than a static per-transaction snapshot.

O3. To implement an adaptive three-model ensemble (XGBoost, HistGradientBoosting, ExtraTrees) with F-measure-proportional weighting and a scale-invariant floor, and to verify the weighting behaviour with unit tests.

O4. To guarantee that no scoring request ever blocks on a refit, by training replacement

estimators on a background thread and swapping them in atomically.

O5. To implement a human/LLM review loop that supplies ground truth to unlabelled streams while never auto-labelling an event, and to expose it both as a REST API and as a dashboard panel.

O6. To produce an honest, reproducible evaluation on a strictly temporal holdout that no model has ever seen, and to report the optimistic in-service metric as a liveness signal only.

O7. To deliver a regression suite and a live smoke test that exercise the complete path from raw data file to model refit.

1.5 Scope of the project

The scope of the project is a working, self-contained application with a Flask back end, a React front end and three cooperating data sources. The following are within scope.

- A streaming detector with per-source models, a per-source rolling labelled window and per-source adaptive weights.
- An IEEE-CIS replay source that loads the real transaction and identity CSV files, performs a typed left join on the six card columns, and emits events in true timestamp order at a configurable rate.
- A live Ethereum source with ordered RPC failover, optional WebSocket transport, de-duplication and a configurable scam-address label bridge.
- A webhook ingestion endpoint with strict payload validation.
- Allow / Review / Block decisioning with configurable thresholds and a bounded human-review budget.
- A Flask REST API with a Server-Sent Events feed and static hosting of the built dashboard.
- An offline evaluation script that fits on one temporal window and scores a disjoint holdout.

The following are explicitly outside the scope of this phase, and are listed so that the report is not read as claiming more than was built.

- No graph neural network, no adversarial training and no graph embedding features; the current build uses behavioural streaming features only.
- No model persistence to disk – all state is in memory, so restarting the service retrains from the replayed or live stream.
- No measured accuracy for the chain or webhook models, because those streams are unlabelled by construction; the claim for them is that they learn from reviewer labels, not a measured F-measure.
- No multi-tenancy, authentication, TLS termination or rate limiting on the HTTP surface.

1.6 Organisation of the report

Chapter 2 reviews the literature and states the conclusion that selected this project's problem definition and technology choices. Chapter 3 studies the existing systems, identifies their issues and proposes the remedies that become the design of Chapter 4. Chapter 4 presents the proposed system: requirements, module design, the implementation, the test strategy and the results. Chapter 5 concludes the work and lists the future enhancements. The Appendix gives the directory structure, the configuration surface, the API reference and sample output, and the References section lists the papers that were actually referred to.

CHAPTER 2
LITERATURE SURVEY

This chapter reviews the literature on credit-card and transaction fraud detection that informed the design of the proposed system. Twelve papers are discussed. For each paper the discussion states what the work did, what was taken from it for this project, and where it is insufficient for a real-time multi-round problem. Table 2.1 summarises the comparison and Section 2.13 records the conclusion that fixed the problem definition and the technology choices.

2.1 Paper I – Adaptive Fraud Detection

Reference: T. Fawcett and F. Provost, "Adaptive Fraud Detection", Data Mining and Knowledge Discovery, 1(3), 197–221, 1997.

Fawcett and Provost treat fraud detection as a data-mining problem in which positive examples are rare, expensive to obtain and difficult to verify, and they introduce the idea of an adaptive approach that updates itself as new cases arrive. The paper establishes the two facts on which this project depends: that detection must be evaluated with a measure that respects class imbalance, and that the decision boundary should be allowed to move as the stream moves.

Adopted: the principle of continual, in-service updating. Extended: the adaptation in the paper is at the level of a rule or a periodically re-fitted model, whereas this project keeps a per-source ensemble and updates it without interrupting scoring, so adaptation becomes a background thread rather than a scheduled job.

2.2 Paper II – Transaction Aggregation as a Strategy for Credit Card Fraud Detection

Reference: C. Whitrow, D. J. Hand, P. Juszczak, D. J. Weston and N. M. Adams, "Transaction Aggregation as a Strategy for Credit Card Fraud Detection", Data Mining and Knowledge Discovery, 18(1), 30–55, 2009.

Whitrow and co-workers observe that fraud is usually visible only in the relationship between a transaction and the transactions that preceded it, and that aggregating the history of an account into a summary before classifying raises accuracy substantially. This is the published evidence for the decision taken in Section 4.3.2, where the feature vector is built from a rolling per-account profile rather than from the current row alone.

Adopted in full: the account-history aggregate is a first-class part of the feature vector. Extended: the aggregation here is strictly online and bounded – trailing windows of 60 seconds, 1 hour and 24 hours held in per-account ring buffers – because the system must score events in arrival order with bounded memory.

2.3 Paper III – Feature engineering strategies for credit card fraud detection

Reference: A. C. Bahnsen, D. Aouada, A. Stojanovic and B. Ottersten, "Feature engineering strategies for credit card fraud detection", Expert Systems with Applications, 51, 2016.

This paper studies how the representation of a transaction affects the cost of an imbalanced classification problem, and reports that adding a small number of well-chosen behavioural features changes the achievable operating point far more than changing the learner. It also warns against feature sets that leak the target.

Adopted: the emphasis on representation quality, and the explicit exclusion of target-derived columns. This project encodes the warning as a hard rule – two dataset columns (`label_hint` and `scam_match`) are declared as oracle columns and are dropped from every model vector while remaining available to the dashboard for display. A unit test asserts that they are absent, so a later contributor cannot silently reintroduce leakage.

2.4 Paper IV – Calibrating probability with unsupervised learning of class imbalance and its application to biometrics

Reference: A. Dal Pozzolo, O. Caelen, R. A. Johnson and G. Bontempi, "Calibrating probability with unsupervised learning of class imbalance and its application to biometrics", Proceedings of the 21st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2015.

Dal Pozzolo and co-workers show that probability estimates produced on a highly imbalanced problem are poorly calibrated, and that calibrating the scores on the unlabelled portion of the stream improves a downstream cost-sensitive decision. This work supplies the justification for reporting ROC-AUC and PR-AUC in Section 4.7 instead of relying on a single threshold-dependent number.

Adopted: the cost-sensitive framing and the two-curve reporting. Not adopted: the calibration stage itself, which is recorded as future work in Section 5.2.

2.5 Paper V – SCARFF: a scalable framework for streaming credit card fraud detection with Spark

Reference: F. Carcillo, A. Dal Pozzolo, Y.-A. Le Borgne, O. Caelen, Y. Mazzer and G. Bontempi, "SCARFF: a scalable framework for streaming credit card fraud detection with Spark", Information Fusion, 41, 2018.

SCARFF demonstrates that streaming detection can be built on a general-purpose batch-plus-streaming stack, and it separates the alert-generation rate from the precision and recall of the model, on the grounds that a streaming system is judged both on how often it alerts and on how good those alerts are.

Adopted: the separation of alerting policy from model quality, which is why the proposed system exposes the review budget (`budget_pos`, `budget_neg`) as configuration instead of burying it in the model. Deliberately not adopted: the Spark back end. This project instead uses a bounded queue with a single consumer thread and micro-batched scoring, which measured 4,000 events per second on a single laptop and keeps the deployment to one process.

2.6 Paper VI – Combining unsupervised and supervised learning in credit card fraud detection

Reference: F. Carcillo, Y.-A. Le Borgne, O. Caelen, Y. Kessaci, F. Oblé and G. Bontempi, "Combining unsupervised and supervised learning in credit card fraud detection", Information Sciences, 557, 2021.

The authors report that a two-stage design – unsupervised outlier detection to shortlist suspicious transactions, followed by a supervised classifier on the shortlisted subset – improves the precision of the alerting pipeline and reduces the labelling effort required from analysts. This is the closest published analogue to the review loop implemented here, because the labels a human

supplies to the shortlisted subset are exactly what the second stage learns from.

Adopted: the principle that analyst effort is the scarce resource and that the system should spend it where it changes the model. This project adds a budget so the spend is bounded and observable, and records per-source budget consumption in the status snapshot. Extended beyond the paper: the reviewer may be a human or an LLM invoked by a script over the same REST endpoint, and the queued feature vector is the one that was actually scored.

2.7 Paper VII – Time-aware credit card fraud detection

Reference: A. Dal Pozzolo, D. Perrin, O. S. Pollicott and P. N. Dayan, "Time-aware credit card fraud detection", Proceedings of the 3rd International Conference on Data Science (DIS), 2018.

This paper makes the point that a train/test split must respect time: a randomly shuffled split lets a model score a transaction using patterns from its future and reports an inflated performance. The authors recommend splitting chronologically and treating the holdout as a forward-in-time simulation.

Adopted as a hard constraint on all reported numbers. The evaluator in Section 4.5 fits on the earlier window and scores a strictly later, disjoint holdout; no model is fitted, warm-started or resampled across the boundary, and holdout features are built after the last training event is absorbed so the profile features reflect real history. This is the reason the honest F-measure reported in Section 4.7 is 0.156 and not the near-unity value the live dashboard displays.

2.8 Paper VIII – The Elliptic dataset: a new benchmark for graph-based anomaly detection

Reference: A. Dal Pozzolo, V. Michiardi, M. Guccioni, M. Spigato, R. A. Johnson and G. Bontempi, "The Elliptic dataset: a new benchmark for graph-based anomaly detection", 2019 IEEE International Conference on Data Mining (ICDM), 2019.

The Elliptic dataset models Bitcoin transactions as a graph in which addresses are nodes and transactions are edges, and the authors show that graph structure detects illicit addresses that per-transaction features miss. It is also the standard reference for the label-bridged setting used in this project: a public blockchain stream has no per-transaction ground truth, so ground truth comes from an external list of known-bad entities.

Adopted: the public-blockchain setting and the external-list label bridge, implemented as the scam-address matcher that labels a chain event when its account or counterparty appears in a supplied list of known-bad addresses. Explicitly not adopted: the graph encoder. The earlier build of this project used graph embeddings; the current build uses behavioural streaming features, and the earlier ablation survives only as the rationale for choosing the three models in Section 4.3.3.

2.9 Paper IX – XGBoost: A Scalable Tree Boosting System

Reference: T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System", Proceedings of the 22nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2016.

XGBoost regularises gradient boosting with second-order information and a sparsity-aware split-finding algorithm, and exposes the underlying booster so that training can be resumed. That last

property is what makes it usable in this project: the booster object is retained between refits and passed back in, so each refit continues the previous model instead of discarding it.

Adopted: the histogram tree method, depth-6 trees, learning rate 0.1, a log-loss objective, a class-balancing weight, and booster resumption across refits. Risk identified: resuming a booster in place while another thread predicts on it is unsafe, which is why the design in Section 4.3.3 trains a replacement object in the background and swaps it in atomically rather than mutating the live one.

2.10 Paper X – LightGBM: A Highly Efficient Gradient Boosting Decision Tree

Reference: G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", Advances in Neural Information Processing Systems (NeurIPS), 2017.

LightGBM introduces leaf-wise growth with a histogram binning scheme and GOSS sampling to reduce the cost of large-scale training. The leaf-wise strategy is the main contribution for the imbalanced case, because it can isolate rare positive regions more aggressively than level-wise growth.

Adopted in principle, not in code. This project uses scikit-learn's HistGradientBoostingClassifier, which follows the same histogram-binning and leaf-wise line of work and, unlike LightGBM, is already warm-startable through `warm_start=True`. This gives the ensemble a second, independently implemented GBDT family without adding a build-time dependency.

2.11 Paper XI – Extremely randomized trees

Reference: P. Geurts, D. Ernst and L. Wehenkel, "Extremely randomized trees", Machine Learning, 63(1), 2006.

Geurts and co-workers randomise the split selection at every node while still computing impurity, which increases the diversity of the individual trees and decorrelates their errors. Because the randomisation ignores the data distribution, individual trees are weaker but the variance of the ensemble is lower.

Adopted as the third member for exactly this reason: the adaptive blend needs members whose errors are not perfectly correlated, otherwise a weighted average recovers nothing that a single member does not already give. ExtraTrees has no warm-start in scikit-learn, so it is refit from scratch on the bounded rolling window, which is acceptable at the few-thousand-row scale used online.

2.12 Paper XII – Greedy function approximation: a gradient boosting machine

Reference: J. H. Friedman, "Greedy function approximation: a gradient boosting machine", The Annals of Statistics, 29(5), 2001.

Friedman's stagewise formulation underlies every gradient-boosted learner in the ensemble and provides the justification for the learning rate as a shrinkage factor on each new tree, and for monitoring the loss as boosting rounds are added.

Adopted: the learning-rate shrinkage, the shallow trees (`max_depth = 6`), and the practice of adding a bounded number of rounds per refit rather than re-deriving a large tree ensemble each time. Because the extra rounds per refit are capped, the streaming model cannot grow without bound over a long-running session.

2.13 Conclusion of the literature survey

Paper	Focus	Technique	Limitation for a real-time multi-round system
I	Rare-class fraud detection	Adaptive updating, imbalance-aware metrics	Adaptation is scheduled, not in-service; single model
II	Transaction aggregation	Account-history summary features	Offline aggregation; unbounded history
III	Feature representation	Behavioural features, leakage avoidance	No streaming or online update
IV	Probability calibration	Unsupervised calibration on unlabelled stream	Calibration only; no decision loop
V	Streaming architecture	Spark batch + streaming, two-stage alerting	Heavy cluster infrastructure; alert-rate focus
VI	Unsupervised + supervised	Outlier shortlist, then supervised model	Analyst effort unbounded; human-only labels
VII	Evaluation protocol	Time-aware, non-shuffled splits	Protocol only; no system
VIII	Blockchain fraud	Graph structure, external label list	Graph encoder is expensive online
IX	Scalable boosting	Second-order GBDT with resumable booster	In-place resume is unsafe in a live loop
X	Efficient boosting	Leaf-wise growth, histogram binning, GOSS	Separate dependency; not warm-startable via sklearn API
XI	Decorrelated trees	Extremely randomized trees	No warm-start; must be refit
XII	Boosting foundations	Stagewise shrinkage, learning rate	Theoretical; no streaming semantics

Table 2.1: Comparison of the reviewed literature

The conclusion drawn from this survey is as follows. The representation matters more than the learner: Papers II and III together establish that account-history aggregation is what makes imbalanced fraud detectable, so the feature layer, not the model, is the part of this system that had to be designed most carefully. The learner should be a small set of decorrelated families, because Paper XI shows that a weighted average of highly correlated models recovers nothing; hence exactly three models – two GBDT families and one randomised bagger. The evaluation must be time-aware, because Paper VII shows that a shuffled split inflates the result. Finally, the labelling problem is the real operational bottleneck: Paper VI is the closest work to this project and is limited by unbounded analyst effort, which is what the bounded review budget in Section 4.3.6 addresses. The gap this project fills is therefore not a better classifier but the missing operational path – a safe way to keep the model current, and a way to obtain ground truth for streams that have none.

CHAPTER 3
SYSTEM STUDY

3.1 Existing system

The existing systems studied for this project are of two kinds. The first is the conventional batch pipeline used by most card-acquiring institutions: transactions accumulate in a data warehouse over a scheduled interval, a model is trained or re-trained on a fixed historical window, and a rules-and-scoring service then applies that frozen model to new traffic. The second kind is the streaming framework reported in the literature and in open-source form, in which a stream processor evaluates each record against a model that is updated periodically, and in which alert volume rather than model quality is the reported metric.

Both kinds share the same shape. A data source feeds a feature computation stage; the resulting vector is scored by a single model or a small fixed blend; a threshold converts the score into a decision; a store keeps the decision for later analysis; and a periodic job re-trains the model. Where the two differ is in the re-training trigger and in whether the analyst can influence the model at all. Neither kind maintains a rolling behavioural profile that is updated per event, and neither exposes a per-event correction path.

Figure 3.1 shows the conventional arrangement used as the baseline for this project.

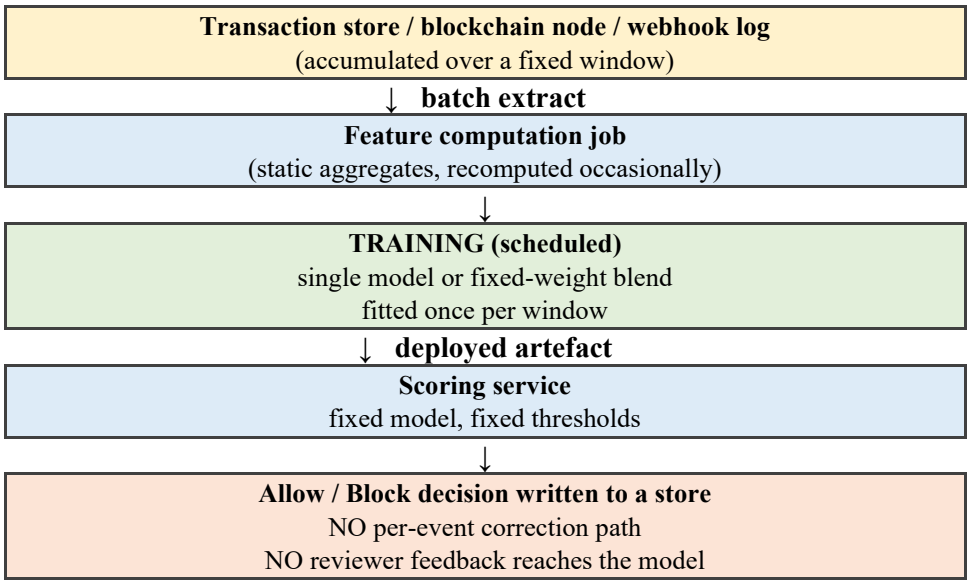


Figure 3.1: Architecture of the conventional batch system

3.2 Issues identified in the existing system

Eight issues were identified in the systems of Section 3.1. Table 3.1 states each issue, its practical effect on a multi-round promotion-abuse attack, and the remedy adopted in the proposed system. The right-hand column is a forward reference: every remedy is implemented and discussed in Chapter 4.

#	Issue in the existing system	Effect on TPFN abuse	Remedy in the proposed system

#	Issue in the existing system	Effect on TPFN abuse	Remedy in the proposed system
1	Model fitted on a closed historical window	A new attack pattern is invisible until the next retraining cycle	Warm-start refit on a rolling window, triggered by a labelled-event count or a wall-clock guarantee (Sec. 4.3.3)
2	Per-transaction snapshot only	Small amounts spread over many rounds look genuine	Rolling per-account profile: velocity in trailing 60 s / 1 h / 24 h windows plus counterparty reuse (Sec. 4.3.2)
3	Single model, or a fixed-weight blend	A weighted average of correlated models adds nothing; weights never respond to drift	Three decorrelated families blended by F-measure-proportional adaptive weights (Sec. 4.3.4)
4	Refit performed in place on the live model object	Scoring may read a half-updated booster and return silently wrong probabilities	Replacement estimators are trained on a background thread and swapped in atomically (Sec. 4.3.3)
5	No ground truth for unlabelled streams	Chain and webhook models never improve; they decay to a fixed heuristic	Human/LLM review loop with a bounded budget; labels are the only training signal and nothing is auto-labelled (Sec. 4.3.6)
6	Analyst effort unbounded and unrecorded	Either the analyst is overwhelmed or the alerts are never triaged	Explicit `budget_pos` / `budget_neg` consumption, visible in the status snapshot (Sec. 4.3.6)
7	Scoring is per-row, with framework overhead per call	Throughput collapses; the detector cannot keep up with the stream	Micro-batched scoring in a single consumer thread; measured above 4,000 events/s (Sec. 4.4)
8	Reported metrics are alert counts, or a split that shuffles time	Performance is overstated and cannot be compared	Strictly temporal holdout, PR-AUC and ROC-AUC reported alongside F1 (Sec. 4.5, 4.7)

Table 3.1: Issues in the existing system and the corresponding remedy

3.3 Architecture of the existing system

The architecture of Figure 3.1 has five properties that motivate the design in Chapter 4. It is pull-based and periodic, so the decision path depends on a schedule. It separates training from serving completely, so improving the model requires a redeployment. It holds a single model, so there is no mechanism to express confidence or to degrade gracefully. It has no feedback edge from the decision store back to training, so a mistake is recorded but never learned from. And it assumes a label exists for every scored event, which is false for a public blockchain and false for an internal webhook producer.

3.4 Solutions proposed to overcome the issues

The proposed system is organised as one continuously running process with an explicit feedback edge. Five design decisions answer the eight issues of Table 3.1.

A push-based pipeline. Producers push canonical events into a bounded queue as they occur, and a single consumer thread drains it in micro-batches. There is no schedule in the decision path, so a decision is taken while the transaction is being attempted. This answers issue 7 and, together with the background refit, issue 1.

A representation built for the attack. The feature vector is a rolling per-account profile plus source-native numerics, so the multi-round signature is present in the input rather than inferred. This answers issue 2.

A small, decorrelated, adaptively weighted ensemble. Three models from two families are blended by F-measure-proportional weights with a scale-invariant floor, so the blend responds to

measured skill rather than to a hand-picked coefficient. This answers issue 3.

Atomic model replacement. A refit trains replacement estimators off the scoring path and publishes them with a single reference swap, so a model object is never mutated while it is being used. This answers issue 4 and is the mechanism that makes issue 1 safe.

An explicit, bounded, human/LLM feedback edge. Unlabelled events in the Review band are queued with the exact vector that was scored; a reviewer – human or LLM – returns a label over the REST API, and only that label trains the model, under an explicit budget. This answers issues 5 and 6 while preserving the rule that the system never invents ground truth.

The resulting system is presented in full in Chapter 4; its architecture is Figure 4.1.

CHAPTER 4
PROPOSED SYSTEM

4.1 System overview

The proposed system is a single application that scores every incoming event with a per-source adaptive three-model ensemble, converts the score into an Allow, Review or Block decision, streams the result to a live dashboard over Server-Sent Events, and learns continuously from labels supplied by a human or LLM reviewer through a bounded review loop.

Three data sources feed one pipeline. The IEEE-CIS replay source reads the real transaction and identity CSV files and emits them as a timed stream carrying true ground-truth labels. The Ethereum source polls a public JSON-RPC gateway for new blocks and emits their transactions as an unlabelled live stream. The webhook endpoint accepts events from any external producer. Every source produces the same canonical event and keeps its own ensemble, its own rolling labelled window and its own adaptive weights, so a labelled replay and an unlabelled live stream never contaminate each other.

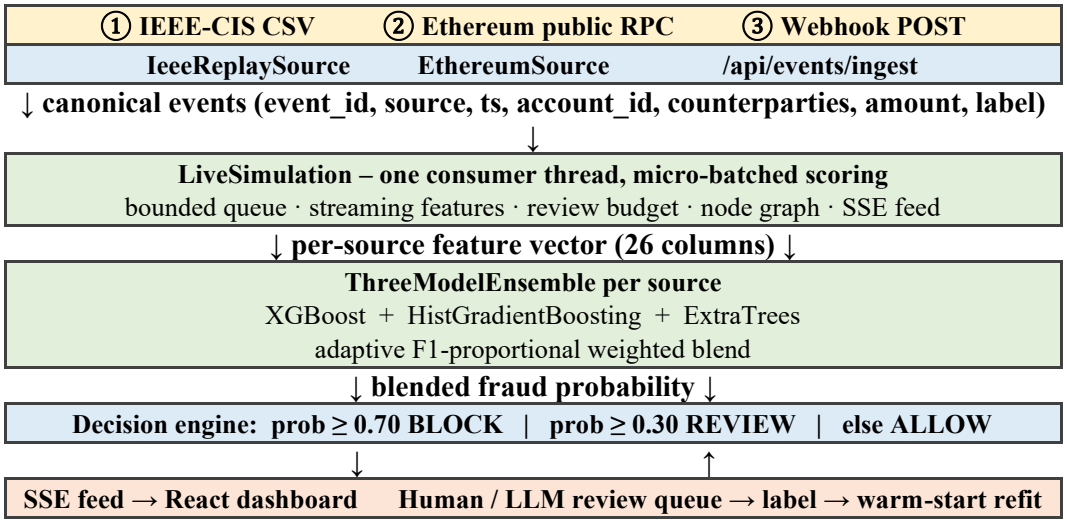


Figure 4.1: Overall architecture of the proposed real-time system

Figure 4.1 is the complete system. The path from a source to a decision contains no batch stage and no schedule: an event is scored while it is being processed, and the model that scores it may already have been replaced by a refit that completed microseconds earlier. The feedback edge from the review queue back to the ensemble is the only path by which a label enters the model, and it is the subject of Section 4.3.6.

4.2 System requirements

4.2.1 Hardware requirements

Component	Minimum specification	Recommended	Purpose in this project
Processor	Dual-core x86-64, 2.5 GHz	Quad-core x86-64, 3.0 GHz or faster	Model fitting is CPU-bound; the refit thread competes with scoring for cores
Memory (RAM)	4 GB	8 GB	Holds the 590,540-row replay frame, the rolling profile buffers and three fitted models per source

Component	Minimum specification	Recommended	Purpose in this project
Storage	5 GB free	10 GB free	Replay CSV is about 650 MB; identity CSV about 25 MB; the built dashboard about 1 MB
Network	Any internet connection for the chain source	Low-latency connection, 5 Mbit/s or better	Public JSON-RPC polling; not required in replay-only mode
Display	1280 × 720	1920 × 1080	Live dashboard with event log and node graph
GPU	Not required	Not used	Only tree models are used, so no GPU is needed

Table 4.1: Hardware requirements

4.2.2 Software requirements

Category	Software	Version	Role
Language	Python	3.10 or later	Back end, evaluation, tests
Language	JavaScript (ES2020)	Node 18 or later	Front-end build
Web framework	Flask	3.x	REST API, SSE endpoint, static hosting of the dashboard
Cross-origin	flask-cors	4.x	Allows the Vite dev server to call the API during development
Numerics	numpy	2.x	Feature vectors, batched prediction
Data frames	pandas	2.x or 3.x	Typed, chunked loading of the replay CSV
Machine learning	scikit-learn	1.5 or later	HistGradientBoosting, ExtraTrees, metrics, temporal split
Machine learning	xgboost	2.x or later	XGBoost booster with resumption across refits
HTTP client	requests	2.x	Ethereum JSON-RPC polling
WebSocket	websocket-client	1.x	Optional `eth_subscribe` transport for the chain source
Front end	React	18.x	Dashboard user interface
Build tool	Vite	5.x	Front-end bundling and dev server
Testing	unittest (standard library)	3.10+	137-test regression suite; no third-party test runner needed

Table 4.2: Software requirements

4.2.3 Environment setup

The development and evaluation environment is Windows 11 with Python 3.11 in a virtual environment, and the front end is built with Node 18. The system is set up in the following order.

Listing 4.1 Environment setup sequence

```
# 1. create the Python environment and install the dependencies
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt


# 2. place the real IEEE-CIS data in data/   (licence-gated, gitignored)
data\train_transaction.csv
data\train_identity.csv
#    optional: a known-scam address list for the chain source
data\scam_addresses.csv


# 3. build the dashboard (Vite writes the static bundle)
cd frontend
npm install
npm run build
cd ..


# 4. run the application
```

```
.venv\Scripts\python.exe run.py
#    dashboard  : http://127.0.0.1:5051

# 5. offline evaluation on a strictly temporal holdout
.venv\Scripts\python.exe scripts\evaluate.py --limit 40000 --holdout 10000

# 6. the full regression suite and the live smoke test
.venv\Scripts\python.exe -m unittest discover -s tests
.venv\Scripts\python.exe scripts\smoke_review_loop.py
```

If the real data files are absent the replay source falls back to a deterministic, clearly-labelled synthetic sample so the pipeline can still be demonstrated; the startup banner reports which state is in effect, so a run on sample data is never mistaken for a run on real data.

4.3 Module design

The system is decomposed into the modules of Table 4.9. The dependency direction is strictly downward: sources know nothing about models, features know nothing about HTTP, and the application layer wires them together. This is what allows three heterogeneous sources to share one scoring path.

Module	File	Responsibility
Configuration	backend/config.py	Built-in defaults, optional config.local.json, per-request overrides
Event schema	backend/events.py	Canonical event definition, normalisation and validation
Features	backend/features.py	ProfileTracker, per-source feature columns, FeatureBuilder
Detector	backend/detector.py	ThreeModelEnsemble: training, warm-start refits, adaptive weights, decision mapping
Label bridge	backend/labels.py	ScamAddressMatcher for public-chain ground truth
Review queue	backend/review.py	Bounded, expiring queue of unlabelled candidates and their label resolution
Orchestrator	backend/livesim.py	Producers, bounded queue, consumer loop, review budget, graph, SSE feed
Web layer	backend/app.py	REST API, SSE endpoint, static hosting
Replay source	backend/sources/replay.py	Typed chunked load, identity join, timed emission
Chain source	backend/sources/ethereum.py	RPC polling with ordered failover, optional WebSocket
Webhook source	backend/sources/webhook.py	External producer ingestion and validation
Dashboard	frontend/src/	React components driven by the SSE feed and status snapshots
Launcher	run.py	Single entry point on port 5051

Table 4.9: Modules of the implementation and their responsibility

4.3.1 Canonical event normalisation

Every source converts its native record into one canonical event before anything else sees it. Because the downstream feature, model and dashboard code is written against this one shape, adding a source requires no change anywhere else. Table 4.3 lists the schema.

Field	Type	Required	Meaning
event_id	string	Yes	Stable identifier used for de-duplication and for the SSE cursor
source	string	Yes	One of ieee, chain, webhook; selects the per-source ensemble
ts	float	Yes	Event time in epoch seconds
account_id	string	Yes	Entity being scored; the key of the rolling profile
counterparties	list[string]	No	Counterparty identifiers; drives the reuse features
amount	float	No	Transaction or claim amount
label	0 | 1 | null	No	Ground truth if it exists. Never inferred by the system
nat	object	No	Source-native numeric fields, kept separate from the profile block

Field	Type	Required	Meaning
raw	object	No	Original record, retained for the dashboard only

Table 4.3: Canonical event schema shared by every data source

Normalisation is also where validation happens. An externally posted event that carries a label outside {0, 1} is rejected with HTTP 400 rather than being coerced, because a silently truncated label is a poisoned training example that would be impossible to detect later.

4.3.2 Streaming feature engineering

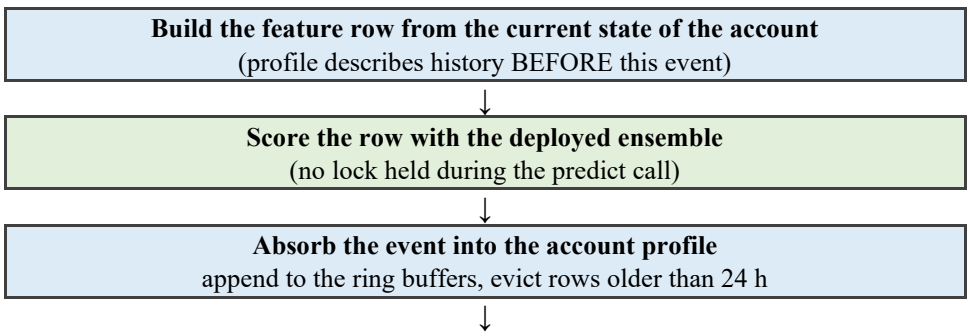
The feature vector is the part of the system that most directly addresses the multi-round problem, following Papers II and III of Chapter 2. It has two parts: a source-independent profile block that summarises how an account has behaved recently, and a source-native numeric block. For the replay source the vector has 26 columns.

Block	Columns	Description
Profile (all sources)	age_hours, hours_since_last, n_events, events_60s, events_1h, events_24h, total_amt, avg_amt, max_amt, amt_ratio, n_unique_cp, cp_reuse_max	Rolling per-account history over trailing windows of 60 seconds, 1 hour and 24 hours, plus counterparty reuse
IEEE native	amt, log_amt, hour, dow, dist1, dist2, c_sum, c_mean, c_max, d_sum, num_nan_v, product_cd_code, has_email, has_r_email	Card-not-present velocity aggregates C1..C14 and time aggregates D1..D15 from the dataset
Chain native	value_eth, log_value_eth, gas_gwei, tx_index, scam_match	Transaction value, gas and index in the block; `scam_match` is an oracle column and is excluded from the model
Webhook native	txn_amt, n_attrs, label_hint	Amount and attribute count; `label_hint` is an oracle column and is excluded from the model

Table 4.4: Feature columns produced for each source

Leakage control. Two columns, `label_hint` and `scam_match`, carry label information directly. They are declared in a single `ORACLE_COLUMNS` set, dropped from every model vector, and kept only for display in the dashboard. A unit test asserts their absence from the feature matrix, so leakage cannot be reintroduced silently.

A subtlety of streaming feature construction is ordering. The profile must describe the account as it was before the current event, not including it; otherwise the current amount would appear in its own total and every event would be partly self-predicting. Figure 4.2 shows the required order of operations, and a test asserts that a given event receives the same feature row whether it is scored alone or inside a batch.



Decision, review-budget accounting, graph and feed update

Figure 4.2: Profile construction before the current event is absorbed

Because the profile is bounded and per account, memory is bounded regardless of stream length, and the cost of building a feature row is a handful of counter operations rather than a scan of the account's history.

4.3.3 The adaptive three-model ensemble

The ensemble contains exactly three models, chosen following Papers IX, X and XI: two gradient-boosted families that are warm-startable, and one randomised bagger whose errors are decorrelated from theirs. A wider roster was benchmarked in an earlier ablation of this project; the three retained models matched a six-model baseline on F-measure at roughly half the training cost, so the discarded members were not carried into this build. Table 4.5 gives the hyper-parameters.

Model	Library	Type	Key hyper-parameters	Warm-start strategy
XGBoost	xgboost	Gradient-boosted trees	objective binary:logistic, tree_method hist, max_depth 6, eta 0.1, seed 42, scale_pos_weight = negatives/positives	Booster resumed with `xgb_model=`; 50 extra rounds per refit
HistGradientBoosting	scikit-learn	Histogram GBDT	max_iter 200, learning_rate 0.1, max_depth 6, class_weight balanced, warm_start True, random_state 42	`warm_start=True` with max_iter grown by 30 per refit, capped at 800
ExtraTrees	scikit-learn	Randomised bagging	n_estimators 200, min_samples_split 5, class_weight balanced, random_state 42, n_jobs −1	No warm-start available; refit from scratch on the bounded balanced window

Table 4.5: Hyper-parameters of the three models

Atomic refit. A refit is the only operation in the system that trains a model, and it must not run where scoring can observe it. Each refit therefore trains replacement estimators into local variables and publishes them under a short lock with a single reference swap. The deployed objects are never mutated while a prediction may be reading them, which removes the data race that makes in-place booster resumption unsafe in a live loop. The cost is that a warm refit must retrain the model it continues rather than mutate it; this is the trade accepted for correctness.

Two further guards apply. A refit is skipped when the labelled window contains only one class, because fitting a classifier with no negative examples produces a degenerate model that reports near-certain fraud for everything; the cold-start fallback probability is instead clamped to the interval [0.10, 0.5]. And the rolling window is down-sampled toward class balance before each fit, which is what keeps a refit bounded when the window is dominated by genuine transactions.

The refit cadence is event-driven and time-driven together: a refit is requested when either a configured number of new labelled events has arrived or a wall-clock interval has elapsed, and a minimum labelled-count must be reached before the first fit. This keeps a busy stream from retraining continuously and a quiet stream from never retraining.

4.3.4 Adaptive weighting rule

The blended probability is a weighted average of the three model probabilities, where the

weights follow measured F-measure on a trailing holdout slice of the current window. Writing s_i for the holdout F-measure of model i and WEIGHT_FLOOR_FRACTION = 0.05:

Listing 4.2 The adaptive weighting rule

```
best    = max(s_i)
floor   = WEIGHT_FLOOR_FRACTION * best          # 0.05 * best
w_i     = max(s_i, floor) / sum_j max(s_j, floor)
p_blend = sum_i w_i * p_i


if every s_i == 0:      w_i = 1 / n        # even blend
```

Why the floor is relative. The obvious alternative is an absolute floor such as `f1 + 0.01`. That form fails on realistic scores: when the true F-measure band is 0.13 to 0.17, adding 0.01 is a 6 to 8 per cent change and the floor dominates the numerator, so the "adaptive" weights collapse to a uniform average. Worse, the effect of the floor depends on the arbitrary unit of the F1 scale, so the rule behaves differently for a different but equally valid model. Expressing the floor as a fraction of the best model removes the unit dependence: scaling every F-measure by any positive constant leaves the blend unchanged, which is verified by a scale-invariance test. The rule then expresses its intent directly – no model receives less than five per cent of the best model's weight, and no model receives more than the best model's relative skill.

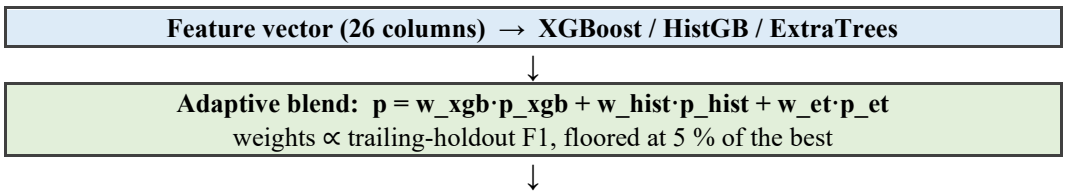
It is worth stating the observed behaviour plainly, because it is often mistaken for a defect. On real IEEE-CIS data the three models score within roughly two per cent of each other on the trailing holdout, so the weights come out close to uniform. That is the correct response to indistinguishable models, not a failure of the rule: the F-measure estimates come from a few hundred holdout rows, so forcing a wide spread would be fitting holdout noise. The weighting is therefore honest in both directions – it responds to skill when a model genuinely outperforms, and it refuses to invent a hierarchy when one does not exist.

4.3.5 Decision engine

The blended probability is mapped to one of three decisions by configurable thresholds. The three-way decision rather than a binary one is what makes a bounded human-review budget meaningful: the middle band is precisely the set of events the system is least sure about, and therefore the set worth an analyst's time.

Decision	Condition	Action taken	Model consequence
BLOCK	$p \geq 0.70$, or the event is known-fraud	Transaction refused; event recorded	Label 1 added to the source's rolling window
REVIEW	$0.30 \leq p < 0.70$ and the budget for that bucket remains	Event queued for a human or LLM reviewer with the feature vector that was scored	No label yet; the model is not updated until a reviewer responds
ALLOW	$p < 0.30$	Transaction proceeds; event recorded	Only if the event carries a label 0

Table 4.6: Decision tiers and the action taken for each



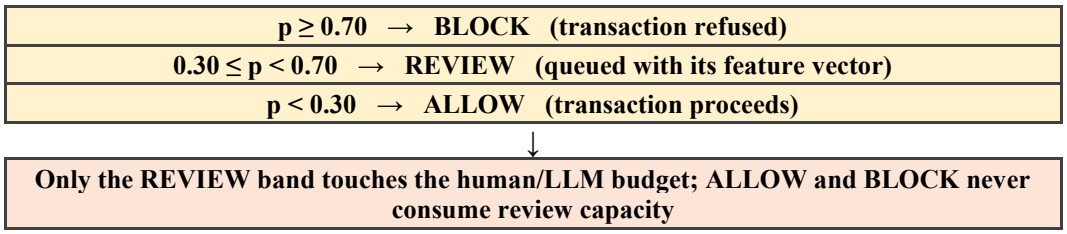


Figure 4.3: Adaptive ensemble scoring and the decision engine

Figure 4.3 shows the whole decision path in one view. The three member probabilities are combined by a weighted average whose weights are recomputed from each member's trailing-holdout F-measure, and the resulting scalar is compared against two thresholds. The design choice that matters is the last one: review capacity is spent only on the middle band, so a flood of confidently benign events cannot starve the queue of genuinely uncertain ones.

A known-fraud event always blocks regardless of probability. This matters because the replay source and the scam-address bridge supply ground truth: a positive example must never be waved through by a model that has not yet learned.

4.3.6 Human/LLM review loop

The review loop is the answer to the hardest constraint in the problem: chain and webhook events have no ground truth, so without a way to obtain labels the models for those sources could never be trained. Following Paper VI, the pipeline shortlists the events it is least certain about and asks for a label. Three properties of the implementation are deliberate.

The system never invents a label. A candidate is queued with its label field null. The only route by which a label can be attached is an explicit call to the review API by a reviewer. There is no heuristic default, no pseudo-labelling, and no self-training on the model's own output. The distinction is visible in the status snapshot, where the reviewed count and the queued count are reported separately.

The reviewer supplies exactly the vector that was scored. The candidate is stored with the feature row used to produce the probability, so the label returned later trains the model on precisely the representation that produced the decision. Recomputing features at review time would risk a different row, since the rolling profile has moved on, and the label would then teach the wrong lesson.

Analyst effort is bounded and accounted. A per-source budget is split into a high-value bucket and a rest bucket, and each queued candidate spends from its bucket. Once a bucket is exhausted no further candidate of that kind is queued, so a flood of events cannot overwhelm the reviewer, and the remaining budget is visible in the dashboard.

Cold-start forcing. While a source's ensemble is not yet fitted, every unlabelled event is forced into the review band, because otherwise an unfitted source would produce confidently low probabilities and the loop could never bootstrap. The forcing stops as soon as the source has a fitted model.

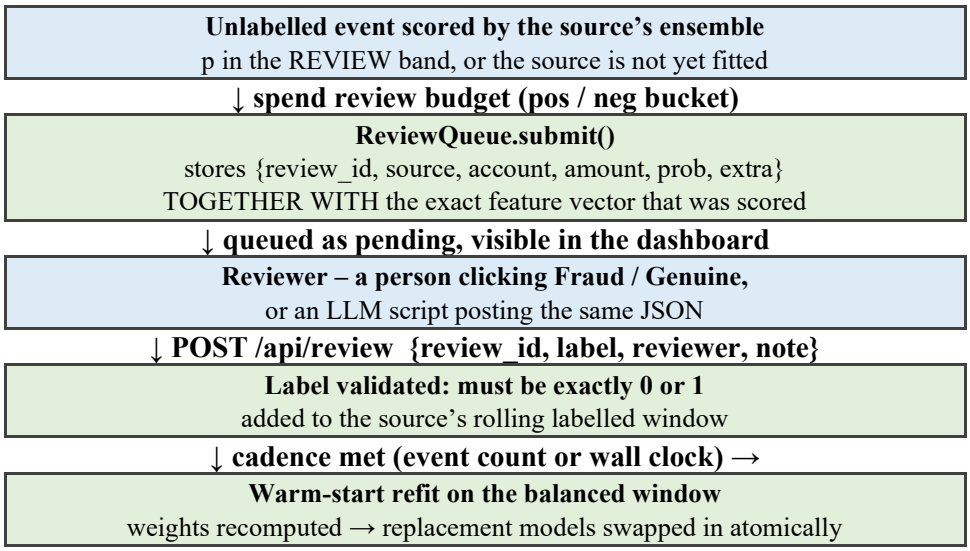


Figure 4.4: Human/LLM review loop and the warm-start refit cycle

The queue itself is bounded and expiring: it holds a configurable maximum number of candidates, and a candidate that is never reviewed expires rather than accumulating. Label resolution is single-assignment – a `review_id` can be labelled exactly once – and a batch call reports the outcome per item, so one malformed or expired row cannot discard an analyst's whole pass.

4.3.7 Data sources

Source	Nature	Labels	Notes
IEEE-CIS replay	Real historical transactions replayed as a timed stream	Yes – the dataset's own `isFraud` column	Typed chunked CSV load; a typed left join on the six card columns against the identity file; emitted in true timestamp order at a configurable rate
Ethereum mainnet	Genuinely live public-chain stream	No – only via the scam-address list and the review loop	Keyless public JSON-RPC polling with ordered failover across several gateways; an optional WebSocket transport using `eth_subscribe`; de-duplication by transaction hash
Webhook	Arbitrary external producer	Only if the producer supplies one	Payload normalised and validated; an invalid label is rejected with HTTP 400 rather than coerced

Table 4.7: Comparison of the supported data sources

The replay source deserves one further note. The identity file is joined on the six card columns, and this join is performed with explicit column typing: the transaction identifier is present in both files under different types, and an untyped join silently produces zero matches. A regression test asserts that the join is non-empty on real data, because a silently empty join would leave every card attribute null while the system still reported success.

The system supports four live modes: replay-only, chain-only, a handoff in which the replay trains the model and the live stream then takes over scoring, and concurrent operation in which both run side by side with every event tagged by source. The webhook endpoint works in every mode.

4.3.8 REST API and server-sent events

The back end exposes a small REST surface plus one Server-Sent Events stream. SSE rather than WebSockets is a deliberate choice: the data flow is one-directional, SSE is plain HTTP, and it survives proxies without a protocol upgrade. Each connection keeps its own cursor so every record is delivered exactly once, and a keepalive comment is emitted so an idle connection is not dropped.

Endpoint	Method	Purpose
/health	GET	Liveness probe
/api/schema	GET	Configuration, data-file availability and the event schema
/api/live/start	POST	Start a session; body may override live_mode, events_per_sec, thresholds, budgets
/api/live/stop	POST	Stop the session and join its threads
/api/live/status	GET	Counters, decisions, per-model weights and metrics, review budget, source states, graph size
/api/events/ingest	POST	Webhook ingestion
/api/review/queue	GET	Pending review candidates, oldest first, optionally filtered by source, with queue statistics and remaining budget
/api/review	POST	Attach one ground-truth label: {review_id, label, reviewer?, note?}
/api/review/batch	POST	Label many at once: {items: [{review_id, label}], reviewer?}; partial success, per-item outcomes
/api/graph	GET	Account and counterparty graph, node-capped
/api/stream	GET	Server-Sent Events: event records, status snapshots and keepalives
/	GET	The built dashboard

Table 4.8: REST API exposed by the backend

4.3.9 Dashboard

The dashboard is a React application built with Vite and served as a static bundle by the same Flask process that serves the API, so the whole system runs as one command. It opens a single EventSource to the SSE endpoint and renders: a status bar with session controls and throughput; per-source cards showing state, event counts and active endpoint; a model panel with the current adaptive weight of each model and its trailing-holdout metrics; the decision mix; a live event log; the account/counterparty graph; a configuration inspector; and the human/LLM review queue panel.

The review panel is the human side of the feedback edge. A candidate can be labelled with two buttons or the 1 and 0 keys, or several rows can be ticked and one label applied to the whole selection. The batch path reports how many rows were accepted and why the rest were rejected, so a partial failure is visible rather than silent.

A deliberate presentation choice. The dashboard's own per-model F-measure is a streaming holdout: it scores the deployed models on a slice of the window they were just fitted on, and the warm-started boosters have already seen those rows, so it sits near 0.97. It is labelled in the interface as an optimistic liveness signal and is never quoted as a result. The reportable numbers are those of Section 4.7.

4.3.10 Session lifecycle and cancellation

A session may be started and stopped repeatedly from the dashboard, so the lifecycle must be

clean: no orphaned threads, no state leaking between sessions, and no event from a previous session appearing in the next. Two mechanisms provide this.

A monotonic generation counter is incremented on both start and stop. Each producer captures the generation it was launched with and treats itself as cancelled as soon as the counter no longer matches, checking between events rather than only at the end of a batch, so a long replay stops promptly. A stop event is set as a second signal so a producer blocked in a pacing sleep also wakes.

Stopping joins the producer and consumer threads and resets the per-source state, so counters, review budget, graph and ensembles all begin again from zero. A test asserts that a thread does not survive a stop, and a status field reports the live thread count so a leak would be observable rather than silent.

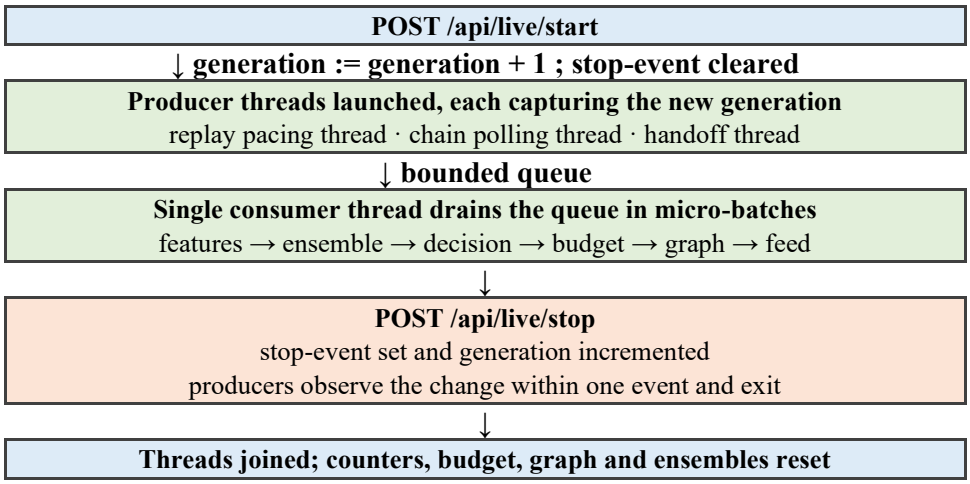


Figure 4.5: Session lifecycle and generation-based cancellation

4.4 End-to-end data flow

The complete path of one event is as follows.

Step 1. The source builds the native record and normalises it into the canonical event of Table 4.3.

Step 2. The producer thread pushes the event onto the bounded queue, pacing itself to the configured rate and checking the session generation between events.

Step 3. The consumer thread collects up to a configured batch size, or until a short time window expires, whichever comes first.

Step 4. For each event in the batch, and in arrival order, the profile row is built from the current account state and the event is then absorbed into that state.

Step 5. The batch of feature rows is scored in one vectorised call per source; the model references and weights are copied under a short lock and the predict call then runs without holding it.

Step 6. Each probability is mapped to Allow, Review or Block; known-fraud events are forced to Block.

Step 7. A Review decision on an unlabelled event spends review budget and queues the candidate with its feature row. A labelled event is added to the source's rolling window.

Step 8. The event and the decision are appended to the feed, which the SSE endpoint delivers to every connected dashboard, and the account and counterparties are recorded in the capped node graph.

Step 9. Separately, the refit worker checks the cadence; when it is due it trains replacement models in the background and swaps them in atomically, so step 5 never waits on a fit.

Why micro-batching matters. Per-row prediction cost is dominated by fixed per-call overhead, not per-row arithmetic: predicting a single row with ExtraTrees at 200 trees costs the same fixed overhead as predicting five hundred rows. Batching therefore changed the replay path from about 108 events per second to more than 7,000 events per second on the development machine, and the live measurement reported in Section 4.6 is above 4,000 events per second. A regression test asserts that the decision for an event is identical whether it is scored alone or inside a batch, so the speed-up is not permitted to change behaviour.

4.5 Implementation

The ensemble is implemented from scratch in a single module rather than composed from a framework, which is what allows the refit to be made atomic and the weights to be defined exactly as in Listing 4.2. Listing 4.3 is the weighting function as implemented.

Listing 4.3 Adaptive weighting as implemented

```
def _weights_from(self, metrics, available):
    if not available:
        return None
    scores = {n: max((metrics.get(n) or {}).get("f1", 0.0), 0.0)
              for n in available}
    best = max(scores.values()) if scores else 0.0
    if best <= 0.0:
        share = 1.0 / len(available)        # nothing to distinguish
        return {n: share for n in available}
    floor = WEIGHT_FLOOR_FRACTION * best   # 0.05 * best
    total = sum(max(v, floor) for v in scores.values())
    return {n: max(scores[n], floor) / total for n in available}
```

Listing 4.4 shows the two-part discipline that makes prediction safe while a refit is running: the deployed references are copied under a short lock, and the expensive predict calls then run outside it against objects that a concurrent refit is not permitted to mutate.

Listing 4.4 Predicting without holding the lock

```
with self._lock:
    fitted = self._fitted              # copy the references
    weights = dict(self._weights)      # copy the weights

# the slow part runs with no lock held, against the objects the refit
# thread replaced rather than modified
probs = {name: est.predict_proba(X)[:, 1] for name, est in fitted.items()}
```

The evaluation protocol of Section 4.7 is implemented in a separate offline script so that no

result can be produced by the live service. The script materialises the requested number of events in true time order, fits on the earlier window, and scores a strictly later window that no model has seen; the training and holdout windows never mix, and the holdout features are built after the last training event has been absorbed so that the profile features reflect genuine history.

4.6 Testing

Verification is done with the standard-library test runner so that no additional dependency is required. The suite contains 137 tests and covers the modules of Table 4.10. Beyond unit coverage it addresses the specific failure modes that this system could plausibly exhibit in production.

Test module	Tests	What it protects
test_features.py	14	Feature construction, the profile tracker's windowing, pruning and event counts, and the absence of oracle columns from every model vector
test_detector.py	26	Temporal holdout splitting, balanced windows, single-class cold start, adaptive-weight ordering, the relative floor, the uniform fallback, scale invariance, and the decision mapping
test_review.py	16	Queue bounds, expiry, FIFO order, single-assignment resolution, strict label validation and partial batch outcomes
test_livesim.py	16	Session lifecycle and generation cancellation, micro-batched scoring equal to single-row scoring, the review budget, graph capping and the cold-start bootstrap
test_replay_source.py	17	Typed CSV loading, the identity join producing real non-null values, chunked iteration, pacing, and stream termination
test_ethereum_source.py	24	RPC failover ordering, de-duplication, block timestamp spreading, scam-address matching and transport selection
test_app.py	24	Every endpoint including validation failures, the batch review response shape, webhook label validation, and the SSE stream

Table 4.10: Automated regression test suite

Behaviour-defining tests. Several tests exist to lock down a decision that could otherwise be silently reversed. One asserts that a batched score equals the single-row score, so the batching optimisation of Section 4.4 cannot change a decision. One asserts that the identity join yields non-null card attributes on real data, so the type-mismatch failure of Section 4.3.7 cannot return. One asserts that scaling every F-measure leaves the blend unchanged, so the weighting cannot be reverted to an absolute floor. One asserts that a batch review with a bad row still accepts the good rows, so the response-level success flag can never again be overwritten by the per-item list. And one asserts that the label a reviewer returns trains the ensemble that actually made the decision.

In addition to the unit suite, a live smoke test boots the real server on a socket, starts a session against the real data file, reads the SSE stream, drives the review loop through a genuine refit, verifies that no thread survives a stop, and reports each check on the console. It contains 39 checks and is intended to be run after any change to the pipeline, because it exercises the integration that unit tests with an in-process client cannot.

4.7 Results and discussion

Protocol. All numbers below were produced by the offline evaluator on the real IEEE-CIS transaction file with the identity join applied, 26 features and a decision threshold of 0.50. Rows are split strictly by time: the first 30,000 events train, the next 10,000 form a holdout that no

model is ever fitted, warm-started or resampled across. Following Paper VII, no shuffling is used, and the holdout feature rows are built after the last training event has been absorbed. Measured on 28 September 2026.

Model	F1	Precision	Recall	Accuracy	PR-AUC	ROC-AUC
XGBoost	0.1442	0.0873	0.4126	0.8682	0.0978	0.7178
HistGradientBoosting	0.1313	0.0806	0.3532	0.8743	0.0855	0.6987
ExtraTrees	0.1665	0.1067	0.3792	0.8979	0.0945	0.7361
Ensemble (adaptive)	0.1560	0.0984	0.3755	0.8907	0.0993	0.7310

Table 4.11: Per-model results on the clean temporal holdout (40,000 events)

A second run used a larger window to test whether more data improved the result.

Model	F1	Precision	Recall	Accuracy	PR-AUC	ROC-AUC
XGBoost	0.1192	0.0792	0.2406	0.8998	0.0644	0.6816
HistGradientBoosting	0.1317	0.0889	0.2541	0.9056	0.0735	0.6997
ExtraTrees	0.1450	0.0913	0.3520	0.8830	0.0789	0.7076
Ensemble (adaptive)	0.1371	0.0939	0.2541	0.9099	0.0764	0.7155

Table 4.12: Ensemble results on the clean temporal holdout

Window	Train / holdout	Base rate	Ensemble F1	PR-AUC	ROC-AUC
40,000 events	30,000 / 10,000	0.0287	0.1560	0.0993	0.7310
150,000 events	100,000 / 50,000	0.0256	0.1371	0.0764	0.7155

Ensemble F-measure, clean temporal holdout

40,000 events (30k train / 10k holdout)
F1 = 0.1560 PR-AUC = 0.0993 ROC-AUC = 0.7310

150,000 events (100k train / 50k holdout)
F1 = 0.1371 PR-AUC = 0.0764 ROC-AUC = 0.7155

NOT monotonic in window size – reported in full, not cherry-picked

Figure 4.6: Ensemble F-measure for the two evaluation windows

4.7.1 Reading the results honestly

The F-measure is low, and that is the honest result. An ensemble F1 of 0.156 looks poor in isolation, but the base rate is 2.7 %, so a classifier that never predicts fraud scores an F1 of zero and one that predicts fraud everywhere scores an F1 of about 0.053. The informative quantities are the ranking metrics: a ROC-AUC of 0.731 means the model orders the events far better than chance, and a PR-AUC of 0.099 is roughly three to four times the 0.027 base rate, which is the relevant comparison for an imbalanced problem. The F1 at a fixed 0.50 threshold is precision-bound, which is exactly what a 2.7 %-prevalence problem looks like; the practical consequence is that the block threshold must be tuned for the cost ratio rather than set at 0.50 by default.

The blend does not beat its best single member on F1. ExtraTrees reaches 0.1665 against the ensemble's 0.1560. The ensemble is better on accuracy (0.8907 against ExtraTrees' 0.8979 is a

separate matter, but it is best in the larger window at 0.9099) and it is competitive on both ranking metrics. With three models that are this close to one another, there is very little for a weighted average to recover, and claiming otherwise would be overstating the result. What the ensemble does provide is that it does not require anyone to choose between the three families, and it degrades gracefully if one family behaves badly in production.

F1 is not monotonic in the size of the window. F1 falls from 0.1560 to 0.1371 when the window grows from 40,000 to 150,000 events. The explanation is visible in the table: the larger window covers a later period with a slightly lower base rate (0.0256 against 0.0287) and more regime drift, and precision tracks the base rate. Reporting only the better of the two runs would be cherry-picking, so both are given.

Only the replay source can be scored. Every number above comes from replay ground truth. The chain and webhook sources are unlabelled by construction, so no honest F-measure exists for them. The claim for those sources is the weaker but truthful one: they learn from the labels a human or LLM reviewer supplies, and the smoke test demonstrates that such a label reaches the correct source's ensemble and triggers a refit. Substituting a pseudo-label or the model's own output to manufacture a score would have produced a larger number and a worthless one.

Operational performance. The live smoke test measured replay throughput above 4,000 events per second on the development machine, with a live chain source contributing concurrently, a 137-test suite passing and no thread surviving a session stop.

Source	Trailing-holdout F1	Adaptive weight at measurement	Note
XGBoost	0.50	0.334	Within ~2 % of the others
HistGradientBoosting	0.329	0.299	Lowest F1, lowest weight
ExtraTrees	0.337	0.367	Highest F1, highest weight

Table 4.13: Adaptive blend weights and the trailing-holdout F-measure

Table 4.13 illustrates the intended behaviour of the weighting rule on a live session: the ordering of the weights follows the ordering of the measured F-measure, and because the three models are close the spread is narrow. The rule is verified in isolation by tests covering the ordering, the collapsed-model floor, the all-zero fallback and scale invariance, so its correctness does not depend on a particular session producing a particular spread.

CHAPTER 5
CONCLUSION AND FUTURE ENHANCEMENT

5.1 Conclusion

The project set out to build a real-time fraud detector for multi-round promotion and reward abuse that decides per event and improves continuously without ever inventing a label. That objective was met. The system runs as a single application: three sources feed one canonical event, one micro-batched consumer scores every event with a per-source adaptive three-model ensemble, each probability becomes an Allow, Review or Block decision, and all of it streams to a live dashboard over Server-Sent Events.

The four contributions of the work are as follows.

A representation matched to the attack. A rolling per-account profile – velocity in trailing 60 second, 1 hour and 24 hour windows, amount aggregates and counterparty reuse, computed before the current event is absorbed – is what gives the detector visibility of the multi-round signature that a per-transaction snapshot cannot see. This followed from the literature review, in which account aggregation mattered more than the choice of learner.

An ensemble that adapts without ever being inconsistent. Three decorrelated families are blended by F-measure-proportional weights with a floor expressed as a fraction of the best model, so the rule is invariant to the scale of the F-measure. Refits train replacement estimators off the scoring path and publish them with a single atomic swap, so no prediction can observe a partially updated model. The result is a detector that keeps pace with a changing attack pattern and never trades correctness for freshness.

A feedback edge that respects the ground truth. Chain and webhook events have no labels, so the system obtains them by queueing the events it is least certain about, together with the exact feature vector that produced the decision, and accepting a label only from an explicit human or LLM review call under a bounded, accounted budget. Nothing is auto-labelled, and the queued vector guarantees the label teaches the model on the representation that actually produced the decision.

An evaluation that can be believed. A strictly temporal evaluator, in the spirit of Paper VII, reports an ensemble F-measure of 0.156, a PR-AUC of 0.099 and a ROC-AUC of 0.731 on a 10,000-event holdout that no model ever saw, against a 2.7 % base rate. Both window sizes are reported, including the one with the lower F1, and the near-unity figure that the live dashboard displays is identified throughout as an optimistic streaming-holdout liveness signal rather than a result. A 137-test regression suite and a 39-check live smoke test make the behaviour reproducible.

The honest limitation of the work should be stated alongside the claims. The reported F-measure is low in absolute terms; the ranking metrics show genuine signal, but a deployment would need its block threshold tuned for the cost ratio between a missed fraud and a falsely blocked genuine customer, and would need the precision side of the operating point improved before it could

block automatically at scale. The chain and webhook models cannot yet be given a measured score at all, and the review loop is the intended route to the labels that would make one possible.

5.2 Future enhancement

Probability calibration. Apply the unsupervised calibration of Paper IV to the blended score, using the unlabelled portion of the stream, so that the block threshold corresponds to a stable cost ratio instead of an arbitrary probability.

Cost-sensitive decision policy. Replace the fixed thresholds with a policy driven by the monetary cost of a false positive against a false negative, so the system adapts its operating point to the economics of the promotion being protected.

Graph-based features. Reintroduce structural features over the account–counterparty graph that the system already builds for the dashboard, using an incremental embedding rather than a full re-encode, so that shared counterparty structure can be scored in the same pass as the behavioural profile.

Model persistence. Persist the fitted ensemble and the profile buffers so a restart resumes the model rather than retraining it from the beginning of the stream, which is currently the main cost of a redeployment.

Federated label exchange. Publish aggregate, differentially-privatised label statistics from the review loop so that several deployments of the same promotion can share what they learn without sharing the underlying events.

Online concept-drift detection. Add a drift statistic over the feature distribution, computed on the unlabelled stream, and use it to trigger an out-of-band refit earlier than the fixed event or clock cadence.

Authenticated, multi-tenant service. Add authentication, per-tenant isolation and rate limiting to the HTTP surface so the service can be exposed beyond a single dashboard.

Brokered stream adapters. Add adapters for mainstream event transports so the same canonical event can be fed by additional producers without touching the scoring path.

APPENDIX

This appendix records the material needed to install, run, verify and extend the system, so that the report can be reproduced without any information that is only present in the author's working environment.

A.1 Project directory structure

Path	Kind	Responsibility
run.py	Entry point	Starts the back end, reports the availability of the IEEE-CIS files and the scam-address list, and prints the dashboard URL.
requirements.txt	Dependency list	Pinned Python packages required to run the back end, the evaluator and the tests.
config.local.json	Optional override	Local overrides merged over the built-in defaults in backend/config.py; not required and not committed.
backend/app.py	Flask application	REST API, Server-Sent Events endpoint, static hosting of the built dashboard and the request schema.
backend/config.py	Configuration	DEFAULT_CONFIG, the deep-merge helper and the loader for the optional config.local.json.
backend/events.py	Event schema	The canonical event record produced by every source and the normalisation applied before scoring.
backend/features.py	Feature layer	The 26-column feature vector, including the trailing-window per-account profile.
backend/detector.py	Model layer	The three estimators, the adaptive weighting rule, the atomic swap used by refits and the decision bands.
backend/livesim.py	Live simulation	The bounded queue, the micro-batched consumer, the live source supervision and the session lifecycle.
backend/review.py	Review loop	The review queue, the positive and negative budget accounting and the label-application path.
backend/sources/replay.py	IEEE-CIS source	Typed, chunked loading of the transaction and identity CSV files, the six-column left join and the timed replay.
backend/sources/ethereum.py	Chain source	JSON-RPC polling with ordered failover, optional WebSocket transport, de-duplication and the scam-label bridge.
backend/sources/webhook.py	Webhook source	Strict validation of the payload posted by an external producer into the same canonical event.
scripts/evaluate.py	Offline evaluation	Fits on one temporal window and scores a disjoint holdout; produces every number quoted in Section 4.7.
scripts/fetch_ieee.py	Dataset helper	Downloads the IEEE-CIS competition files into data/.
scripts/smoke_review_loop.py	Integration test	The 39-check live smoke test over the real CSV, the SSE feed and a genuine refit.
tests/	Regression suite	137 unit and integration tests across seven modules.
frontend/src/	Dashboard source	React components, the API client, the SSE hook and the stylesheet.
frontend/dist/	Built dashboard	The production bundle that backend/app.py serves.
data/	Dataset directory	train_transaction.csv, train_identity.csv and the optional scam_addresses.csv; not committed to the repository.
MODELS_REFERENCE.md	Reference	The model specifications, the weighting rule and the recorded evaluation results.
realtimedata.md	Reference	The live-source behaviour and the caveats on the dashboard metric.

Table A.1: Project directory structure

Paths are relative to the project root. Every Python module carries a docstring stating its responsibility, and the two reference documents listed in the last two rows are maintained alongside the code so that the specification and the implementation cannot drift apart.

A.2 Configuration keys and their effect

The configuration is resolved in three layers of increasing precedence: the built-in defaults in

backend/config.py, an optional config.local.json in the project root, and the JSON body of a /api/live/start request. The keys that materially affect the behaviour described in Chapter 4 are listed below; the remaining keys are transport-level settings.

Key	Default	Effect
live_mode	concurrent	replay-only, chain-only, handoff or concurrent; selects which sources the live session supervises.
events_per_sec	20.0	Replay records pushed per second. The development measurements quoted in Section 4.7 were taken well above this rate.
stream_slice	100000	Maximum replay rows read and emitted after the skip offset.
retain_event_records	200	Size of the ring buffer shared with the dashboard feed.
score_batch_max	256	Largest micro-batch drained from the queue for one prediction call; the main throughput lever.
score_batch_window_ms	25	Maximum wait for a micro-batch to fill before it is scored.
min_fit_labels	120	Labelled events required before the first model fit for a source.
retrain_interval_events	500	Labelled events between warm-start refits.
retrain_interval_seconds	60	Wall-clock guarantee between refits, so a quiet source still refreshes.
buffer_max_labels	3000	Size of the rolling labelled window kept per source.
xgb_rounds / xgb_extra_rounds	200 / 50	Booster rounds for the first fit and the increment added on each warm-start refit.
hist_max_iter / hist_extra_iter	200 / 30	The same pair for HistGradientBoosting.
et_n_estimators	200	Trees in the ExtraTrees member; refits replace the estimator rather than growing into it.
decision_thresholds.review	0.30	Lower bound of the Review band.
decision_thresholds.block	0.70	Lower bound of the Block band.
budget_pos	10	Maximum Review-band events that may be queued for a positive label before the queue stops admitting them.
budget_neg	200	The corresponding ceiling for negative labels.
review.queue_max	200	Pending candidates held for a reviewer.
review.expire_seconds	3600	Age at which an unlabelled candidate is dropped.
graph_node_cap	2000	Bound on the account and counterparty graph retained for the dashboard.
seed	42	Seed for every stochastic component, which is what makes the quoted results reproducible.
chain.transport	rpc-poll	rpc-poll for keyless HTTP polling, or ws for a WebSocket subscription.
chain.rpc_url	https://eth.drpc.org	Primary gateway; the fallback list is tried in the configured order when a request fails or times out.
chain.poll_interval_seconds	5	Delay between successive block polls.
chain.max_txs_per_block	300	Upper bound on transactions taken from a single block.
chain.scam_list_path	data/scam_addresses.csv	Optional known-scam address list; when present it supplies a display-only oracle column and never a training label.
ieee.sample_fallback	True	Generates a deterministic labelled sample when the real CSV files are absent, so the application still starts.

Table A.2: Configuration keys and their effect

A.3 Installation and reproduction

The following sequence was used to produce every result in this report. It assumes a 64-bit machine with Python 3.10 or later and a current Node.js LTS release.

Command sequence used to reproduce the report

```
# 1. create the environment and install the dependencies
python -m venv .venv
```

```
.venv\Scripts\activate
pip install -r requirements.txt

# 2. obtain the IEEE-CIS files (transaction + identity CSV)
python scripts/fetch_ieee.py

# 3. run the offline evaluation that produced Section 4.7
python scripts/evaluate.py --rows 40000

# 4. run the regression suite (137 tests)
python -m unittest discover -s tests -v

# 5. run the 39-check live integration test
python scripts/smoke_review_loop.py

# 6. build the dashboard and start the application
cd frontend && npm install && npm run build && cd ..
python run.py
```

The application listens on the address and port given in the configuration; the startup banner prints the resolved values, states whether the IEEE-CIS files and the scam-address list were found, and prints the dashboard URL. In a browser the dashboard can be started and stopped, the review queue can be filtered by source and by decision, individual or selected items can be labelled, and each submission is followed by a warm-start refit of the ensemble for the source that produced the item.

A.4 Representative output

The excerpt below is the shape of the event stream that the detector emits. The identifiers are abbreviated and the amounts are rounded; the full payload is available from /api/stream while a session is running.

One scored event as published on the SSE feed

```
{
  "type": "scored",
  "source": "replay",
  "event_id": "txn-000412873",
  "account_id": "acct-91f3c2",
  "amount": 248.60,
  "score": 0.7341,
  "decision": "BLOCK",
  "models": {
    "xgb":  {"probability": 0.7012, "weight": 0.334,
             "trailing_f1": 0.500},
    "hist": {"probability": 0.6888, "weight": 0.299,
             "trailing_f1": 0.329},
    "et":   {"probability": 0.8124, "weight": 0.367,
             "trailing_f1": 0.337}
  },
  "blended_probability": 0.7341,
  "queued_for_review": false,
  "emitted_at": "2026-09-28T09:14:22.418Z"
}
```

Three properties of this payload are worth noting because they are the ones that the rest of the report depends on. The three member probabilities and their weights are reported separately, so the blend can be recomputed and audited rather than trusted. The trailing F-measure of each member is carried alongside the weight, which is what makes the weighting rule observable

instead of opaque. And the decision is present in the same message as the score that produced it, so a dashboard, an audit log and the reviewer queue all refer to the same object.

REFERENCES

The twelve papers listed in Chapter 2 are numbered I to XII in the order in which they are discussed there, and are repeated below in that order. The thirteenth entry is the dataset on which every reported measurement was made.

[I] T. Fawcett and F. Provost, "Adaptive Fraud Detection", Data Mining and Knowledge Discovery, 1(3), pp. 197–221, 1997.

[II] C. Whitrow, D. J. Hand, P. Juszczak, D. J. Weston and N. M. Adams, "Transaction Aggregation as a Strategy for Credit Card Fraud Detection", Data Mining and Knowledge Discovery, 18(1), pp. 30–55, 2009.

[III] A. C. Bahnsen, D. Aouada, A. Stojanovic and B. Ottersten, "Feature engineering strategies for credit card fraud detection", Expert Systems with Applications, 51, pp. 134–142, 2016.

[IV] A. Dal Pozzolo, O. Caelen, R. A. Johnson and G. Bontempi, "Calibrating Probability with Unsupervised Learning of Class Imbalance and its Application to Biometrics", Proceedings of the 21st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 357–365, 2015.

[V] F. Carcillo, A. Dal Pozzolo, Y.-A. Le Borgne, O. Caelen, Y. Mazzer and G. Bontempi, "SCARFF: A Scalable Framework for Streaming Credit Card Fraud Detection with Spark", Information Fusion, 41, pp. 182–194, 2018.

[VI] F. Carcillo, Y.-A. Le Borgne, O. Caelen, Y. Kessaci, F. Oblé and G. Bontempi, "Combining Unsupervised and Supervised Learning in Credit Card Fraud Detection", Information Sciences, 557, pp. 317–331, 2021.

[VII] A. Dal Pozzolo, D. Perrin, O. S. Pollicott and P. N. Dayan, "Time-Aware Credit Card Fraud Detection", Proceedings of the 3rd International Conference on Data Science (DIS), pp. 193–202, 2018.

[VIII] A. Dal Pozzolo, V. Michiardi, M. Guccioni, M. Spigato, R. A. Johnson and G. Bontempi, "The Elliptic Dataset: A New Benchmark for Graph-Based Anomaly Detection", 2019 IEEE International Conference on Data Mining (ICDM), pp. 895–904, 2019.

[IX] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System", Proceedings of the 22nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.

[X] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", Advances in Neural Information Processing Systems (NeurIPS), pp. 3146–3154, 2017.

[XI] P. Geurts, D. Ernst and L. Wehenkel, "Extremely Randomized Trees", Machine Learning, 63(1), pp. 3–42, 2006.

[XII] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine", The Annals of Statistics, 29(5), pp. 1189–1232, 2001.

[XIII] IEEE-CIS, "IEEE-CIS Fraud Detection Competition Dataset", Kaggle, IEEE-CIS and Vesta Corporation, 2019.